

PostShow App Breakdown

A personal architecture and functionality guide for the current local FastAPI / SQLite concert poster app.

This document is intentionally separate from the website folder. It explains how the app is structured, how data moves through it, and what code systems power the features built so far.

Current app folder	C:\Users\tfitc\Documents\PostShow
Main runtime	Python, FastAPI, SQLite, Jinja templates, CSS, vanilla JavaScript
Database file	concerts.sqlite3

1. Executive Snapshot

PostShow is a local FastAPI and SQLite web application for collecting and sharing concert posters, show memories, and festival attendance. It is currently a server-rendered Python app with a clean HTML/CSS interface, session-based accounts, image uploads, profile pages, feeds, venue and artist pages, and a growing social layer.

The app is intentionally small and understandable. The backend is one FastAPI application file, the persistence layer is an object-oriented repository class around SQLite, and the frontend is built from Jinja templates, one main stylesheet, and one small JavaScript file for image framing, carousels, tabs, and copy/share behavior.

The core product idea is: a user logs in, adds a show or festival, finds or uploads a poster, optionally adds carousel photos and setlist content, and then sees that post appear on their profile, friends feed, and global feed ordered by the actual show date. The latest share-card feature creates a separate visual card for social sharing.

? Backend: FastAPI, Starlette sessions, Jinja2, SQLite, Pydantic models.

? Frontend: server-rendered HTML templates, CSS, plain JavaScript.

? Data: users, show_posts, post_images, friendships.

? Media: uploaded profile photos, banners, posters, and carousel images saved under static/uploads.

? Current working folder: C:\Users\tfitc\Documents\PostShow.

2. Technology Stack

PostShow is not a React or Node app. It is a Python web app that renders pages on the server. Each browser request hits FastAPI, the route function gathers records from SQLite through ConcertRepository, and Jinja2 renders HTML templates using that data.

FastAPI handles HTTP routes, form submissions, file uploads, and JSON API endpoints. Starlette's SessionMiddleware stores login state in signed cookies. SQLite keeps the data in a single local file named concerts.sqlite3. This makes the project easy to inspect, copy, back up, and run locally.

- ? FastAPI: main web framework and API router.
- ? SQLite: local relational database file.
- ? Jinja2Templates: HTML rendering layer.
- ? Pydantic BaseModel: request validation for API create/update endpoints.
- ? Dataclasses: User, ShowPost, and PostImage objects in models.py.
- ? Werkzeug secure_filename: sanitizes uploaded media filenames.
- ? Vanilla JS: no frontend build step, bundler, or npm app runtime.
- ? CSS: light/dark themes, responsive layouts, cards, profiles, share cards, and image frames.

3. Project Folder Structure

The project is intentionally flat. The main app code is in `app.py` and `models.py`. Templates live in `templates/`. Styling and client-side JavaScript live in `static/`. Uploaded images are stored under `static/uploads/`. Tests are in `tests/`.

This layout is simple enough for a beginner to trace: request comes into `app.py`, `app.py` calls methods in `models.py`, the route picks a template, the template pulls styling from `static/styles.css` and behavior from `static/postshow-editors.js`.

```
PostShow/  
app.py FastAPI app, routes, forms, helpers, integrations  
models.py Dataclasses plus ConcertRepository database layer  
schema.sql SQLite table definitions  
poster_search.py Public poster candidate lookup helpers  
requirements.txt Python dependencies  
start-postshow.ps1 PowerShell restart/start helper  
concerts.sqlite3 Local SQLite database file  
templates/  
base.html Shared layout/nav/scripts  
_post_card.html Reusable post card component  
home.html Logged-out landing page  
feed.html Friends/global feed page  
profile.html Profile header and grid/feed view  
add_show.html Create post form  
edit_post.html Edit post form  
post_detail.html Full post page  
share_card.html Social share-card page  
listing.html Band/festival/venue listing pages  
static/  
styles.css Sitewide CSS and responsive design  
postshow-editors.js Drag/zoom, carousel, auth tabs, copy link  
uploads/ Uploaded media files  
tests/  
test_show_date_ordering.py
```

4. Request and Data Flow

A typical page request starts in the browser and ends as HTML. For example, opening `/feed/friends` calls `friends_feed()` in `app.py`. That route requires a logged-in user, asks the repository for friend posts, fetches carousel images and post owners, then renders `feed.html`. `feed.html` loops over posts and calls the `post_card` macro from `_post_card.html`.

The app follows a repeated pattern: route function, repository method, template render. This pattern makes new pages straightforward to add because most of the work is either a new repository query or a new template view.

- ? Logged-out / goes to `home.html` with global post previews.
- ? Logged-in / redirects to `/feed/friends`.
- ? Profiles use `/profile/{username}?view=grid` or `?view=feed`.
- ? Post pages use `/posts/{post_id}`.
- ? Share cards use `/posts/{post_id}/share`.
- ? Data-changing form routes return redirects so page refreshes do not duplicate actions.

```
Browser request
-> FastAPI route in app.py
-> optional require_login() / get_current_user()
-> ConcertRepository method in models.py
-> Jinja template in templates/
-> CSS/JS assets from static/
-> rendered HTML response
```

5. Database Design

The database is SQLite. The app keeps records in four main tables: users, show_posts, post_images, and friendships. The schema lives in schema.sql and is also migration-friendly because ConcertRepository.initialize() calls _ensure_columns() to add missing columns when the app starts.

This design keeps accounts, posts, media, and social graph separated. show_posts owns the main poster and show metadata. post_images holds extra carousel photos. friendships is a many-to-many style table represented by user_id and friend_id rows.

- ? Feeds and profiles order by show_posts.show_date DESC, then created_at DESC.
- ? Older shows added later slot into their historical position instead of becoming artificially newest.
- ? The poster is always stored on show_posts and remains the first visual slide.
- ? Additional carousel images are stored separately in post_images.

Table	Purpose	Important fields
users	Accounts and profile customization	username, password_hash, profile_image_url, banner_image_url, x/y/scale fields
show_posts	Shows and festivals	artist, post_type, venue, city, show_date, poster_url, caption, venue coords, setlist, aspect ratio
post_images	Extra carousel images	post_id, image_url, position, position_x, position_y, scale
friendships	Friend connections	user_id, friend_id, created_at

6. Backend and Routes

app.py contains both page routes and JSON API routes. Page routes render Jinja templates for the browser. API routes expose CRUD operations useful for proving the backend structure and for future frontend/mobile clients.

Authentication is session based. `signup()` creates a user, hashes the password with PBKDF2, stores `user_id` in the session, and redirects to the new profile. `login()` verifies the hash and sets the same session field. `require_login()` protects routes like friends feed, account editing, and show creation.

? Core pages: `/`, `/feed/global`, `/feed/friends`, `/profile/{username}`, `/posts/{post_id}`.

? Account pages: `/signup`, `/login`, `/logout`, `/account`.

? Social pages: `/friends`, `/profile/{username}/friend`, friends feed.

? Entity pages: `/bands/{artist}`, `/festivals/{festival_name}`, `/venues/{city}/{venue}`.

? Post management: `/add`, `/posts/{post_id}/edit`, `/posts/{post_id}/delete`.

? Share cards: `/posts/{post_id}/share`.

? API CRUD: `GET/POST /api/posts`, `GET/PUT/DELETE /api/posts/{post_id}`, `GET /api/users`, `GET /api/users/{username}`.

7. Object-Oriented Database Layer

models.py is the object-oriented core. It defines dataclasses that represent app concepts and a ConcertRepository class that centralizes all SQLite access. Instead of scattering SQL throughout templates or random helper code, the route functions call repository methods.

This gives the app a clean learning structure: User, ShowPost, and PostImage are Python objects; ConcertRepository is the one object responsible for creating, reading, updating, and deleting those records.

- ? User: account identity, display name, bio, profile/banner images, image framing data.
- ? ShowPost: artist/festival, venue, dates, poster, caption, setlist, aspect ratio, venue coordinates.
- ? PostImage: extra carousel image records with per-image framing data.
- ? ConcertRepository: initializes schema, handles migrations, executes SQL queries, and returns dataclass objects.
- ? get_yearly_show_number(): calculates a user's show number within a calendar year for share-card stats.
- ? list_friend_posts(): includes both the logged-in user's posts and posts by users they follow/add.

8. Frontend Templates and UI Composition

The frontend is server-rendered with Jinja. `base.html` defines the shell: document head, stylesheet, navigation, theme initialization, shared JavaScript, and clickable-card behavior. Other templates extend `base.html` and fill the content block.

The most important reusable piece is `_post_card.html`. It is a macro that displays a post consistently across feeds, profiles, listings, and detail pages. It renders the owner avatar, artist/festival link, venue link, carousel, social action row, date, caption, and owner menu when appropriate.

- ? `home.html`: simple logged-out homepage with auth panel and global previews.
- ? `feed.html`: one-column social feed for friends/global posts.
- ? `profile.html`: profile banner/avatar plus grid or feed view toggle.
- ? `listing.html`: shared page for bands, festivals, and venues.
- ? `add_show.html` and `edit_post.html`: forms for show/festival creation and updates.
- ? `post_detail.html`: larger post view with full detail metadata.
- ? `share_card.html`: separate visual share-card surface, not the same as post cards.

9. Frontend Behavior, Images, and Styling

`static/postshow-editors.js` provides the interactive behaviors without a frontend framework. Its central idea is direct image framing: images are shown inside fixed frames, and the user can drag to reposition, scroll to zoom, or pinch on pointer devices. Hidden inputs store x, y, and scale values so the backend can save the exact framing.

`static/styles.css` controls the entire visual system: green/black brand palette, light/dark mode variables, responsive navigation, feed cards, profile banners, poster aspect ratios, carousel controls, post action icons, account editor previews, venue map previews, and share-card styling.

- ? Image framing fields: `position_x`, `position_y`, and `scale`.
- ? Poster aspect ratios: 3:4, 2:3, and 1:1.54.
- ? Carousels: poster first, uploaded photos next, optional setlist slide last.
- ? Profile grid: compact Instagram-like poster wall that hides headers/body/actions.
- ? Profile feed: larger post cards in a one-at-a-time scroll style.
- ? Share card: dedicated story-shaped graphic using poster background, profile chip, stat pill, optional carousel image tile, and show metadata.
- ? Theme: `localStorage` remembers light/dark mode.

10. Integrations, Current State, and Next Steps

PostShow currently supports a few optional integrations and demo-friendly fallbacks. Poster lookup avoids private APIs by searching public pages and extracting preview images. Venue lookup can use Google Maps when `GOOGLE_MAPS_API_KEY` is set, but falls back to OpenStreetMap/Nominatim for local demo use. `setlist.fm` lookup is disabled until `SETLISTFM_API_KEY` exists, while manual setlist entry remains available.

The project is now a real local prototype with backend storage, accounts, CRUD, feeds, profiles, media uploads, image framing, social pages, entity pages, and share-card experiments. The biggest technical improvement to make next is to move the project into its own virtual environment and make `start-postshow.ps1` use `.venv` instead of the Codex runtime cache.

- ? Dependency hardening: create `.venv`, install requirements, add missing `jinja2` and `itsdangerous` to `requirements.txt`.
- ? Social depth: make like/comment buttons functional with database tables.
- ? Share cards: add true PNG export/download instead of screenshot-only sharing.
- ? API maturity: add auth-aware API permissions and response schemas.
- ? Data quality: normalize artists, festivals, and venues into separate tables later if the app grows.
- ? Deployment: move from local SQLite/dev server to hosted app, managed storage, and environment secrets when ready.